

ckpttn: Multilevel Circuit Partitioning

Wai-Shing Luk

Abstract—Balanced hypergraph partitioning is a fundamental step in VLSI physical design, and the Fiduccia-Mattheyses (FM) local search, refined inside a multilevel framework, is its workhorse. This paper describes **ckpttn**, an open-source multilevel circuit partitioner built on three principles: keep the FM machinery but make the gain bookkeeping cheap, coarsen the hypergraph aggressively with a primal-dual minimum maximal matching and MinHash duplicate-net pruning, and certify small instances exactly with a middle-levels Gray-code enumeration. We state the partitioning problem, derive the FM gain updates and their bucket data structure, present a simpler greedy refiner for comparison, and describe a symmetry-reduced enumeration that visits every balanced bipartition. On the **p1**, **ibm01**, and **ibm03** benchmarks, multilevel refinement reduces the cut cost by 12-62 percent over flat FM, the greedy refiner is 10-100 times faster but 1.6-2.8 times worse, and the same algorithm behaves consistently across Python, C++, and Rust ports, whose inner loops are all organized around the observation that most gain updates are no-ops. Unlike classical tools, **ckpttn** warns explicitly when the balance constraint cannot be met.

Index Terms—hypergraph partitioning, circuit partitioning, Fiduccia-Mattheyses, multilevel refinement, min-cut.

I. INTRODUCTION

Circuit partitioning divides the cells of a netlist into a small number of roughly equal parts while cutting as few connections as possible. It is the first step of the physical-design flow and reappears throughout it: as the driver of partitioning-based placement, as the basis of hierarchical design and multi-FPGA mapping, and as a load-balancing primitive in parallel computing. Because the number of cut nets directly limits routability and timing, even a few percent of cut reduction is worth pursuing.

This paper describes **ckpttn**, a compact, open-source multilevel partitioner. Its design follows three principles.

1. **Keep FM, but make it cheap.** The Fiduccia-Mattheyses (FM) local search [1] is retained as the refinement operator, but the gain updates are organized so that the common case – a net that stays uncut after a move – does no work, and the inner loop avoids gratuitous allocation and lookup.
2. **Coarsen aggressively.** Large instances are contracted through a primal-dual *minimum maximal matching*, with duplicate nets merged by a MinHash pre-filter, so that the expensive refinement runs on much smaller graphs.
3. **Certify small instances exactly.** Once the coarsest graph is small enough, a middle-levels Gray-code

enumeration visits every balanced bipartition of that graph, turning the multilevel leaf from a heuristic into a certified optimum.

The same algorithm is implemented in Python, C++, and Rust from one specification, and it emits an explicit warning when the requested balance cannot be achieved – a guarantee that widely used tools do not make.

II. PROBLEM FORMULATION

A circuit is modeled as a weighted hypergraph $H = (V, \mathcal{E})$. The vertices V are the modules (cells and I/O pads) and each hyperedge (net) $e \in \mathcal{E}$ connects two or more vertices. A *k-way partition* assigns every vertex to one of k blocks $P_1, \dots, P_k$ that are pairwise disjoint and cover V . A net is *cut* if it has pins in more than one block.

The objective is to minimize the total weight of the cut nets,

$$\min \sum_{e \in \mathcal{E}} w(e) \cdot \mathbb{I}[e \text{ is cut}],$$

subject to a balance constraint on the block weights. With $m(v)$ the module weight and $W = \sum_{v \in V} m(v)$ the total weight, each block must satisfy

$$(1 - \epsilon) \frac{W}{k} \leq \sum_{v \in P_i} m(v) \leq (1 + \epsilon) \frac{W}{k}, \quad i = 1, \dots, k,$$

where the *imbalance factor* ϵ controls how much imbalance is tolerated. Small ϵ (say 0.01) makes the problem much harder; a loose ϵ makes it nearly unconstrained. A few modules may also be *fixed* to a prescribed block (for example, I/O pads), in which case they can never move. Balanced partitioning is NP-hard [8], and for tight tolerance the multilevel method is the practical method of choice.

The choice of method tracks the instance, and it is worth stating the map explicitly. A netlist that is close to a mesh is handled well by spectral bisection; an instance whose balance can be relaxed is a candidate for a network-flow formulation built on the max-flow/min-cut theorem; a graph of at most a few tens of modules can be solved exactly by the enumeration of Section III-D; and beyond that the multilevel method is preferred when the balance tolerance is tight, while single-level FM suffices when it is loose. **ckpttn** exposes the extremes of this map through its presets (Section IV), and the two refiners of Section III are the local-search policies that sit inside the multilevel loop.

III. ALGORITHMS

A. Fiduccia-Mattheyses Refinement

FM is a local search over a fixed partition. It repeatedly moves a single module, always the one whose move improves the objective most, and it accepts worsening moves within a pass so that it can escape shallow local minima. The quantity that drives the search is the *gain* of a move, i.e. the reduction in the cut cost,

$$g(v) = FS(v) - TE(v),$$

where $FS(v)$ (from-side) counts the nets that touch v and lie entirely in v 's current block, and $TE(v)$ (to-side) counts the nets that touch v and lie entirely in the destination block. A positive gain means the cut shrinks.

Because FM considers one move at a time, the relevant quantity after a move is the change in the gain of the moved module's neighbours. For a 2-pin net the update is a simple ± 1 or $\pm w(e)$; for a 3-pin net a short case analysis suffices; a general net only changes a neighbour's gain when the net has at most one pin on one side, so the common case (two or more pins on each side) yields a zero delta. Detecting and skipping that zero is the single most profitable optimization in the whole partitioner.

The candidate moves are kept in a *bucket* priority queue keyed by gain. With integer gains the queue supports insertion, extraction of the maximum, and a key update in $O(1)$ amortized time, which is what makes FM a linear-time heuristic in practice [1]. One *pass* is:

1. initialize the gains of every unlocked module and build the buckets;
2. repeatedly select the highest-gain legal move, execute it, and mark the module *locked* (a locked module cannot move again in this pass), then update the gains of its neighbours;
3. if the queues become empty, or no legal move remains, restore the best partition seen during the pass and start the next pass.

Restoring the best state is done with a snapshot, a re-application of the moves, or a roll-back of the moves; ckptn uses snapshots for the general path and can skip the snapshot entirely when no improving move was found. Passes continue until the cost stops decreasing.

Two properties keep the inner loop cheap. The work done by a move is proportional to the degrees of the incident nets, $\sum_{e \ni v} \deg(e)$, not to the size of the graph; and the candidate set is a bounded priority queue, so with integer gains insertion, maximum extraction, and key update are $O(1)$ amortized. Three pieces of machinery distinguish FM from a plain greedy descent: the bucket queue that makes the best move cheap to find, the per-pass locking that keeps a module from moving twice, and the snapshot/roll-back that makes the temporarily worsening moves safe to take.

Remove all three and one obtains the simpler refiner of the next subsection.

B. A Simpler Refiner

The bucket-and-lock machinery exists to *escape* local minima. If one is willing to accept a worse local optimum, a pure greedy refiner is much cheaper: take the highest-gain move while its gain is positive, with no snapshotting, no rollback, and no locking. We call this the *NN* (no-nonsense) refiner. It monotonically descends to the nearest local optimum, and in return it is 10-100 times faster than FM on flat graphs. The multilevel framework closes most of the quality gap, which is precisely the intended role of a cheap refiner inside the coarsen/ refine loop.

NN is not a separate program but a different policy inside the same pass skeleton. The base class owns the loop – select the maximum-gain move, ask the constraint manager whether it is legal, apply it, update the gains of the neighbours – and exposes a single overridable hook for one pass. FM's hook adds the locking and the roll-back; NN's hook stops at the first non-positive gain. This is the Template Method pattern for the skeleton and the Strategy pattern for the gain and constraint managers, and it is what lets one gain calculator serve the flat, multilevel, and k-way partitioners. Termination of NN is immediate from monotonicity: the cut is a non-negative integer and every accepted move is strictly improving, so the descent cannot cycle and must stop at a local optimum – no locking and no snapshot are required. All three ports implement the same hook with the same threshold, $g_{\max} \leq 0$; a zero-gain move is not taken, because without locking it could cycle.

C. The Multilevel Framework

Flat FM is slow on graphs with hundreds of thousands of modules and is sensitive to the initial partition. The multilevel method removes both problems by solving a hierarchy of successively smaller graphs:

1. **Coarsen.** Contract the hypergraph: find a *maximum matching* of nets that share no module, replace each matched group by a single cluster whose weight is the sum of its members, and add up the weights of any nets that become identical. Repeat until the graph is small.
2. **Initial partition.** Partition the coarsest graph. Because it is small, this step can afford an expensive method (Section III-D).
3. **Uncoarsen and refine.** Project the partition back to the next finer graph and refine it – with FM or NN – at every level on the way up.

The matching is *minimum maximal*: it is maximal (no further net can be added without overlap) and, among maximal matchings, of minimum total weight. It is computed by a primal-dual 2-approximation that maintains a *gap* per net and selects the tightest net in each uncovered neighbourhood. Pairing lighter nets keeps the clusters small

and preserves the structure that the refinement can exploit. Duplicate nets (nets with the same module set) are merged; for low-degree nets this is checked exactly, and for larger nets a 64-element MinHash signature pre-filters clearly dissimilar pairs (estimated Jaccard similarity below 0.8), so that the expensive exact comparison is rarely needed.

Concretely, one contraction step is a five-stage pipeline: compute the min-maximal matching and split the nets into matched clusters and remaining nets, with the unmatched modules forming the cell list; build an intermediate bipartite graph between the cells and the clusters; purge duplicate nets, merging their weights and dropping the self-loops that clustering creates; reconstruct the graph with remapped net indices; and wrap the result in a hierarchical netlist. The hierarchical netlist carries the two projection maps that make the rest of the method work – a node-up map from an original module to its contracted representative, used while coarsening, and a node-down map from a cluster back to one of its members, used while refining. Duplicate detection compares nets of degree at most five exactly and pre-filters larger nets (up to degree 200) with the MinHash signature, falling back to the exact comparison only above the 0.8 similarity threshold.

A recursion guard keeps the contraction honest: a level is contracted only when the result is materially smaller than its parent, $|V^+| \cdot 1.5 < |V|$. Without the guard, a pathological instance can spend a dozen levels shrinking by a few percent each time, as observed on *ibm01*.

D. Exhaustive Refinement by Middle-Levels Gray Code

A balanced bipartition of the coarsest graph can be *enumerated*. Consider assigning each module one bit, 0 for block A and 1 for block B. A balanced partition is a bitstring with exactly n or $n + 1$ ones. The *middle-levels graph* G_n of the $(2n + 1)$ -dimensional hypercube has as vertices all bitstrings of weight n or $n + 1$, with edges between bitstrings that differ in exactly one bit. A Hamiltonian cycle of G_n therefore visits every balanced bipartition exactly once, and each transition is a single module flip – a single FM move. The existence of such a cycle for every $n \geq 1$ was conjectured in 1982 and proved constructively only in 2019 [6]; the construction is memoryless and recursive.

Because a flip touches only the nets incident to one module, the incremental cost update is $O(\deg)$ per step, and the enumeration visits

$$2 \binom{2n+1}{n} = \binom{L}{n} + \binom{L}{n+1}$$

states for L bits. The size grows quickly – 924 states for $n = 11$, 705,432 for $n = 20$ – so the enumeration is used only at the coarsest level, where the contracted graph has at most a few tens of modules. There it yields a certified optimum for that level, which the uncoarsening then refines heuristically.

The construction is memoryless and is organized around a binary tree that encodes the Dyck-path structure of the current bitstring; two recursively defined flip sequences, one per direction of travel, advance the cycle, so the whole traversal is a sequence of single-bit flips. For a balanced bipartition of N modules each of the first N bits is one module’s block assignment; when N is even a single dummy bit is appended so that the bitstring has odd length $L = N + 1$, and the flips that touch the dummy bit are ignored. The leaf threshold follows from the measured cost of the traversal – instant to $n = 11$, about 0.2~s at $n = 20$, and a few seconds at $n = 25$ (10.4~M states) – so the enumeration is kept to the coarsest level, where it certifies the optimum.

E. K-Way Partitioning

For $k > 2$ the search is organized as a sequence of pairwise refinements. Because the exhaustive leaf is inherently two-way, each pair of blocks (i, j) is optimized in turn: the modules currently in block i or j are isolated as the movable set (everything else is treated as fixed), the middle-levels enumeration is run on that pair with the full netlist so that cross-block pins are counted correctly, and the best pair partition is applied. Pairs whose movable set is trivial or larger than a threshold are skipped, since the enumeration is exponential in the number of movable modules. This pairwise sweep is repeated a bounded number of times, and a flat FM legalization keeps the block weights within tolerance between sweeps.

The leaf budget scales with the number of blocks: for k parts the enumeration is allowed $25k/2$ movable modules, spread over the $\binom{k}{2}$ pairs, so each pair sees at most about fifteen modules on average and any pair whose movable set exceeds that is skipped. For $k = 2$ this is the 25-module limit of the previous subsection; for $k = 6$ it is 75 modules over fifteen pairs, which still leaves every pair’s enumeration trivial. The enumeration runs on the full netlist rather than on the isolated pair, so pins that land outside the pair are still counted; this is what makes the pair’s value a correct k -way cost rather than an internal two-way cost.

IV. IMPLEMENTATION

Architecture. The partitioner is decomposed into four collaborating objects, which are instantiated with a shared hypergraph:

- a *gain calculator* that computes gains and their updates, specialized by net degree (2-pin, 3-pin, and general nets, with a high-degree cutoff);
- a *gain manager* that owns the bucket queues and performs select, lock, and key-update;
- a *constraint manager* that decides whether a move is legal, tracking the block weight difference from the ideal;
- a *partition manager* that runs the pass loop, takes snapshots, and restores the best state.

Separating these concerns lets the same gain calculator be reused for the flat, multilevel, and k -way managers.

Presets: balance tolerance and partitioning mode.

Preset	Balance	Mode
default	3%	recursive
quality	1%	direct
highest_quality	0.5%	direct
deterministic	3%	recursive (fixed seed)
large_k	3%	recursive

The arrangement is the Template Method pattern for the pass skeleton – the base class fixes the sequence of select, legalize, apply, and update, and the subclass supplies only the per-pass hook – together with the Strategy pattern for the gain, constraint, and partition managers, which can be swapped without touching the loop.

Ports. The same algorithm is implemented in Python (`ckpttnpy`), C++ (`ckpttn-cpp`), and Rust (`ckpttn-rs`). The Python reference uses NetworkX; the C++ port uses a hand-rolled hypergraph and templates; the Rust port uses `petgraph`. Cross-language regression tests check that a small hand-built partition yields the same cut in all three.

In the Python reference the collaboration is spread over a small set of modules: `FMBiGainCalc` and `FMKWayGainCalc` compute the gains, `FMBiGainMgr` and `FMKWayGainMgr` own the buckets, `FMBiConstrMgr` and `FMKWayConstrMgr` decide legality, `PartMgrBase` fixes the pass skeleton, `FMPartMgr` and `NNPartMgr` are its two policies, `MLPartMgr` drives the hierarchy, `HierNetlist` holds the projection maps, and `min_cover` performs the contraction.

Interface. A command-line front end partitions a hypergraph file directly:

```
ckpttnpy circuit.hgr <k> <epsilon>
```

with options for the input format (hMetis, JSON, DIMACS), fixed modules, output format, a preset, the objective (cut, *km1*, sum of external degrees, or *km1a*), the mode (direct or recursive bisection), the number of parallel starts, and the random seed. Presets bundle the balance tolerance and mode:

Tuning. The presets are thin wrappers over a small set of constants, most of which are fixed at compile time rather than exposed as options. Contraction is attempted while the graph has at least 50 modules; gain updates are skipped on nets of degree above 500; the bucket buffers are sized 32768 for bi-partitioning and 65536 for k-way; and degree-2 and degree-3 nets take a specialized fast path. On the coarsening side, nets of degree at most five are compared exactly for duplication, while larger nets (up to degree 200) are pre-filtered with a 64-element MinHash signature at a Jaccard threshold of 0.8. Several starts can be run in parallel: with *t* starts and a seed *s* the runs use seeds $s + i \cdot 104729$, so the search is reproducible while still exploring different basins; on `ibm02` (19,601 modules) the best single start cost 282 cut nets, whereas four deterministic starts reached 87.

Balance warning. If the requested balance is impossible – for instance, because a single module outweighs an entire block – `ckpttn` reports the failure in its final output. On

a five-module instance whose largest module has weight 10^7 and whose ideal block weight is about 5×10^6 , the block weights are 10^7 and 4 and `ckpttn` prints an explicit warning that the balance constraint is violated, whereas other tools return the same result silently.

Inner loop. Because FM is dominated by the per-move key update, the ports are engineered around the observation that most updates are no-ops. For a general net with at least two pins on each side, moving one pin leaves every neighbour’s gain unchanged, so the update is skipped with a single test instead of a queue operation; in the Python reference this cuts `modify_key` calls on `ibm03` from 7.37~M to 374~K. Two further layers of indirection were removed on the Python side – the NetworkX view objects and the adapter frames around vertex and net lookup – and the adjacency and degrees are now materialised once per gain calculator into plain lists, which together take p1 from 299~ms to 216~ms and `ibm03` from 14.6~s to 8.8~s for identical cuts. The C++ port replaced a freshly allocated gain-change vector per move with a per-calculator buffer returned as a `std::span`, which halves the k-way kernel on `ibm03`; the Rust port removed a clone of the index vector and a `collect` of the neighbours per move, both borrow-checker artifacts, for a uniform 28 percent win. Every change is behaviour-preserving, verified by re-running against the same cut costs, which is the only reason such micro-optimisations are admissible in a reference implementation.

V. EXPERIMENTAL RESULTS

All measurements below are wall-clock times on one host and are intended for relative comparison; the Python numbers in particular are interpreter-bound. Cut costs are hyperedge-cut values (lower is better). Two small benchmarks are used throughout: p1 (833 modules, 902 nets) and `ibm03` (23,136 modules, 27,401 nets, IBM-PLACE format); `ibm01` (12,752 modules, 14,111 nets) is added for the multilevel study. Random starts use a fixed `SplitMix64` stream so that a seed denotes the same assignment in every port.

A. Refiners, Flat and Multilevel

Table I compares the FM and NN refiners, flat and multilevel, for bi-partitioning. Three trends are visible. FM dominates quality: NN is 1.6-2.8 times worse in cut cost. NN dominates speed: flat NN is 10-100 times faster (0.03 s versus 0.31 s on p1; 1.21 s versus 12.96 s on `ibm03`). And the multilevel framework helps both, but helps NN far more – coarsening recovers much of FM’s quality (flat NN 220 to multilevel NN 108 on p1; 7271 to 3922 on `ibm03`), which is exactly why a cheap refiner is useful inside the loop.

The same ordering holds for three-way partitioning. On p1 the mean cut is 346.8 for flat NN and 191.8 for multilevel NN, against 175.4 and 118.8 for FM; on `ibm03` it is 11,109 and 7,230 for NN against 6,470 and 3,627 for FM. Fig. 2

Selected constants of ckpttn.		
Constant	Value	Role
limitsize	50	coarsen while $ V \geq$ this
FM_MAX_DEGREE	500	skip gain updates above this degree
stack_buf_size	32768 / 65536	bounded-queue buffers (bi / k-way)
special_handle_2pin_nets	true	degree-2/3 fast path
LOW_PIN_NET_THRESHOLD	5	exact duplicate check at this degree
MINHASH_SIG_SIZE	64	MinHash signature length
MINHASH_SIMILARITY	0.8	Jaccard pre-filter threshold
MINHASH_MAX_DEGREE	200	skip MinHash above this degree

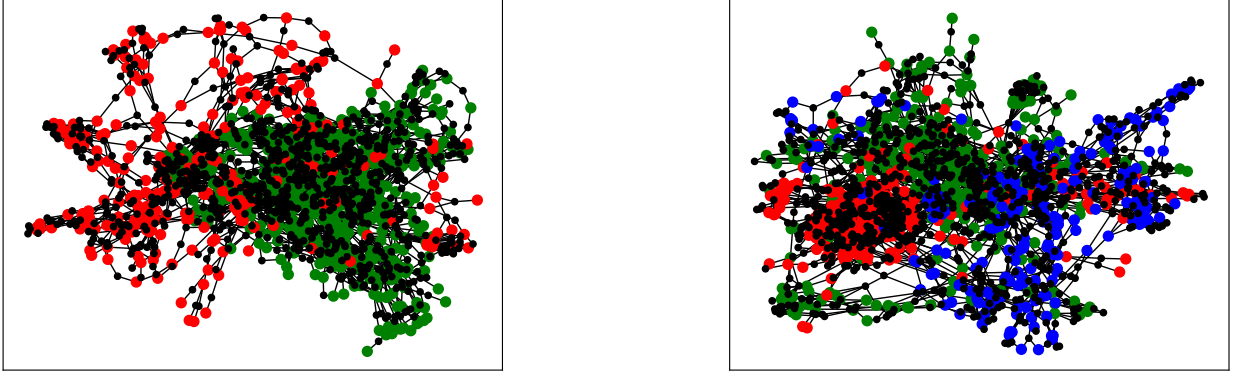


Fig. 1. Two-way (left) and three-way (right) partitions of the p1 benchmark (833 modules, 902 nets), drawn on the bipartite module–net graph with modules colored by block.

shows the full sweep over both benchmarks and all three ports, and Fig. 3 isolates the two effects at work: how much the multilevel framework buys over flat refinement, and how much worse NN is than FM.

B. Multilevel versus Flat FM

Table II isolates the effect of the multilevel framework on ibm01 with $k = 2$ and a balance tolerance of 0.4, over five seeds. Multilevel refinement lowers the cut on *every* run; the average reduction is 20.7 percent, at a cost of about twice the runtime.

The reduction in variance is as important as the reduction in the mean: the one-standard-deviation spread falls from 136.9 to 88.6 cut nets, because the contraction absorbs the noise of the random initial partition.

C. Cross-Language Behaviour

Table III reports the optimize time of the three ports on the same two benchmarks in their release builds. The compiled ports are one to two orders of magnitude faster than the interpreted one; the C++/Rust gap is an implementation detail (the Rust port stores adjacency as a vector-of-vectors and pays a per-call dispatch through an iterator trait), not an algorithmic one, and it narrows as the graph grows.

D. Exhaustive Leaf

Table IV shows the cost of the middle-levels enumeration as a function of the number of modules at the coarsest level. It

is instant up to about twenty modules and remains practical to roughly twenty-five; beyond that the exponential growth dominates and the leaf falls back to FM.

E. Correctness and Warnings

Because the multilevel pipeline is intricate, correctness is checked by cross-validation rather than by the cut cost alone: the ports share a SplitMix64 seed stream and hand-built instances whose cut is known, and the Rust port carries a regression that asserts the multilevel result is non-trivial. The explicit balance warning closes the remaining failure mode – a silently infeasible partition – and makes the tool’s output accountable in an automated flow.

The test surface reflects the same three ports. The reference and its CLI carry 78 tests (25 for the interface and 53 for the algorithm); the C++ suite has 65 cases over the flat, multilevel, and Yosys paths plus the exhaustive leaf; and the Rust port runs 245 tests plus a multilevel regression that asserts a non-trivial result. Cross-port benchmarks re-use the same SplitMix64 stream, so a reported seed denotes the same initial partition in every implementation.

VI. CONCLUDING REMARKS

ckpttn shows that a classical partitioner can be both simple and trustworthy. Keeping FM but making the gain bookkeeping cheap, contracting aggressively with a primal-dual matching and duplicate-net pruning, and certifying the coarsest level by exact enumeration yields a multilevel partitioner whose quality is competitive with mature tools

Cut cost — FM (solid) vs NN (hatched), by language

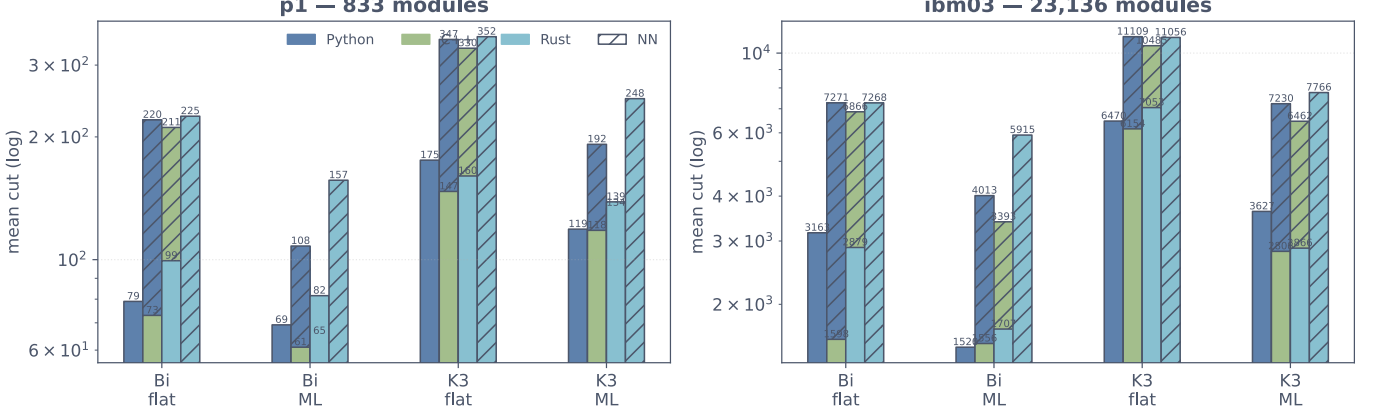


Fig. 2. Mean cut cost of the FM and NN refiners, flat and multilevel, for two-way and three-way partitioning of p1 and ibm03, in the three ports. Lower is better; note the logarithmic scale.

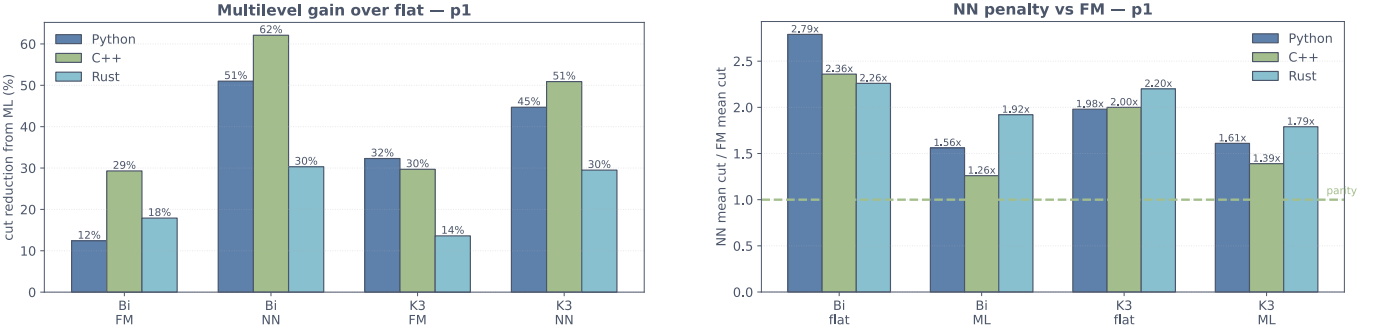


Fig. 3. Left: cut reduction of the multilevel framework over flat refinement (p1, mean). Right: NN penalty, the ratio of NN to FM mean cut, with the parity line shown.

TABLE I

REFINERS ON P1 (833 MODULES) AND IBM03 (23,136 MODULES), BI-PARTITIONING, PYTHON REFERENCE: BEST AND MEAN CUT OVER FIVE SEEDS, AND MEDIAN WALL-CLOCK TIME.

Benchmark	Refiner	Multilevel	Best cut	Mean cut	Median time (s)
p1	FM	no	72	79.0	0.31
p1	FM	yes	56	69.2	1.03
p1	NN	no	211	220.2	0.03
p1	NN	yes	95	108.0	0.81
ibm03	FM	no	2438	3162.8	12.96
ibm03	FM	yes	1240	1519.6	61.71
ibm03	NN	no	6865	7270.8	1.21
ibm03	NN	yes	3265	4012.8	51.47

TABLE II

MULTILEVEL VERSUS FLAT FM ON IBM01 (12,752 MODULES, $k = 2$).

Metric	FM-only	Multilevel	Change
Mean cut	473.4	375.6	−20.7%
Best cut	345	289	−16.2%
Worst cut	709	521	−26.5%
Mean time (s)	14.59	29.97	2.05×

and whose three ports agree. That agreement is not an accident of coding: because the pass skeleton and the gain, constraint, and partition policies are separated, the ports differ only in data layout, and the same separation is what made the inner-loop engineering – the no-op gain updates,

the reusable buffers, the borrowed neighbours – safe to apply uniformly. The experiments confirm the expected trade-offs – multilevel refinement buys 12-62 percent of cut at roughly twice the runtime, and a greedy refiner trades 1.6-2.8 times the cut for one to two orders of magnitude of speed – and the explicit balance warning removes the silent-failure mode that complicates automated tuning and reproducibility.

Natural next steps are a flat (single-array) graph layout in the Rust port to close the remaining gap to C++, a parallel FM refinement, and a systematic study of the contraction guard and leaf threshold across the IBM benchmark family.

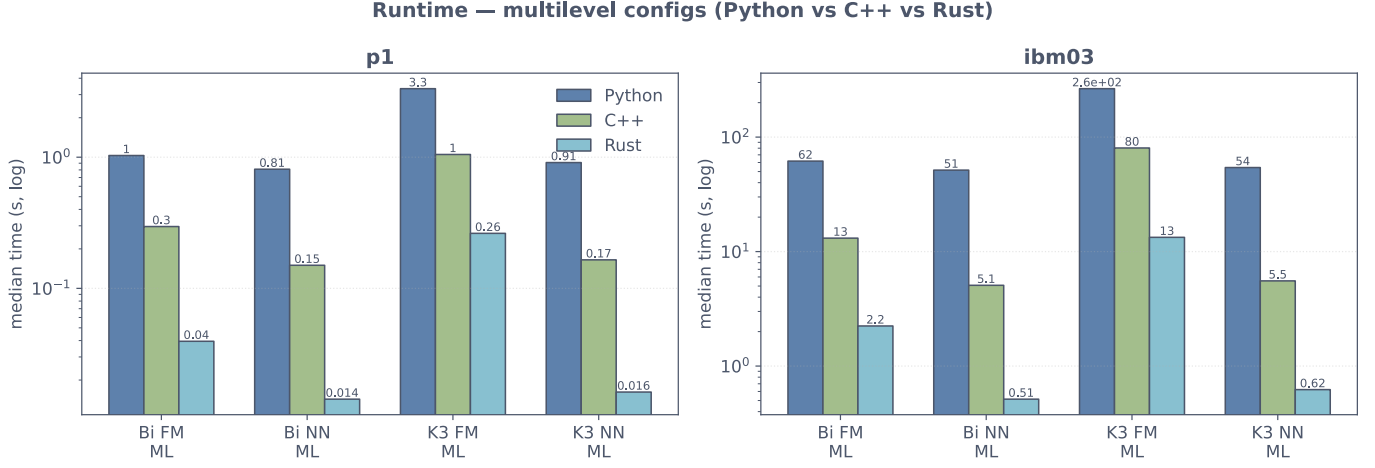


Fig. 4. Median wall-clock time of the three ports on the multilevel configurations, p1 and ibm03 (logarithmic scale).

TABLE III
OPTIMIZE TIME OF THE THREE PORTS (RELEASE BUILDS) ON P1 AND IBM03.

Benchmark	C++ (ms)	Rust (ms)	Python (ms)	C++ / Python
p1 (833 modules, 902 nets)	3.42	51.2	202	59×
ibm03 (23,136 modules, 27,401 nets)	262	1467	9691	37×

TABLE IV
MIDDLE-LEVELS GRAY-CODE ENUMERATION: STATES VISITED AND TIME.

Modules n	States	Time
11	924	< 1 ms
15	12,870	< 1 ms
20	705,432	0.20 s
25	10.4 M	3.4 s

REFERENCES

1. C. M. Fiduccia and R. M. Mattheyses, “A linear-time heuristic for improving network partitions,” in *Proc. Design Automation Conf. (DAC)*, 1982, pp. 175-181.
2. B. W. Kernighan and S. Lin, “An efficient heuristic procedure for partitioning graphs,” *Bell System Technical Journal*, vol. 49, no. 2, pp. 291-307, 1970.
3. G. Karypis and V. Kumar, “Multilevel k-way hypergraph partitioning,” in *Proc. Design Automation Conf. (DAC)*, 1999.
4. S. Schlag et al., “High-quality hypergraph partitioning,” *ACM J. Experimental Algorithmics*, vol. 27, 2023.
5. T. Muetze, J. Nummenpalo, and B. Walczak, “Sparse kneser graphs are Hamiltonian,” in *Proc. ACM Symp. Theory of Computing (STOC)*, 2018.
6. P. Gregor, T. Muetze, and J. Nummenpalo, “The Hamilton cycle problem for the middle levels graph,” 2019.
7. N. Sherwani, *Algorithms for VLSI Physical Design Automation*, 3rd ed. Springer, 1999.
8. A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI Physical Design: From Graph Partitioning to Timing Closure*. Springer, 2011.